

FinCluster

A Comparative Evaluation of Machine Learning Models for Node-Aware MFS Transaction Latency Prediction and Routing

Research Proposal - Version 0.2

Day 1 Revised Research Foundation
7 August 2026

Project: FinCluster AI MVP

Current working direction

Compare multiple regression models for predicting measured transaction service time across controlled node profiles.

Integrate the best-performing model into FinCluster routing and compare it with traditional routing under normal and festival-surge traffic.

Treat node degradation and failure as an optional extension pending mentor approval.

Research status: Proposal only - no measured latency dataset or verified model result exists yet.

Document Control

Field	Value
Document version	0.2
Last updated	7 August 2026
Current stage	Research scope revision and mentor review
Primary ML task	Node-aware service-time regression
Primary target	Measured service_time_ms
External data candidate	PaySim-style synthetic mobile-money transaction data; exact source/version/license must be verified
System evaluation	Winner-model routing vs traditional routing

1. Background

Mobile Financial Services (MFS) platforms process transactions with different data fields, validation requirements, database operations, risk checks, and processing paths. The time required to process a transaction can also vary with the capacity and current condition of the selected processing node.

Traditional routing strategies such as Round Robin distribute requests without estimating how a particular transaction will perform on a particular node. This can lead to poor node selection, avoidable queueing, SLA violations, and inefficient use of heterogeneous infrastructure during normal and burst traffic.

FinCluster AI already provides a simulation environment for comparing AI-assisted routing with legacy routing. The revised research plan extends this system into a controlled experimental framework for transaction-node latency prediction and routing evaluation.

2. Problem Statement

The original research plan treated transactions as fixed Heavy or Light classes. That approach creates a methodological risk when labels are generated from handcrafted rules or assigned solely from transaction type. A model may simply reproduce the rule used to create the label rather than learn independently measured processing behaviour.

The revised problem is therefore to predict the measured processing time of a transaction on a known node profile using pre-routing transaction features, node characteristics, and controlled runtime conditions. The practical research question is whether the most accurate prediction model also improves the performance of the complete routing system.

Important boundary

This proposal does not claim that all banks or MFS providers use one universal internal transaction pipeline.

The study will implement a documented FinCluster reference processing pipeline and report results only for the specified experimental environment.

A new company or hardware environment would require local calibration or retraining.

3. Provisional Research Gap

The provisional gap is that predictive-model quality and end-to-end routing impact are often evaluated separately. This study proposes to compare several latency-prediction models under one controlled transaction-processing environment and then test whether the winner model provides measurable operational improvement over traditional routing baselines.

This gap is not yet a verified novelty claim. It must be refined after a structured literature review and mentor validation.

4. Research Aim

This research aims to compare multiple machine-learning regression models for predicting the processing latency of MFS transactions across controlled node profiles. The best-performing model will then be integrated into the FinCluster routing engine to evaluate whether prediction-based routing improves end-to-end latency, throughput, SLA compliance, and simulated infrastructure cost compared with traditional routing strategies.

5. Research Objectives

- Select and inspect a documented PaySim-style synthetic mobile-money transaction dataset.
- Design and validate a reproducible FinCluster reference transaction-processing pipeline.
- Create controlled node profiles using documented CPU, memory, and concurrency settings.
- Collect repeated service-time and supporting resource measurements.
- Compare several regression models using identical data splits and evaluation protocols.
- Integrate the best model, and optionally the top two models, into the routing simulator.
- Compare ML-based routing with Round Robin, Weighted Round Robin, Least-Loaded, and rule-based routing.
- Evaluate normal traffic and festival-surge traffic, with node failure as an optional extension.

6. Research Questions

RQ1 - Model-level performance: Which machine-learning regression model most accurately predicts Mobile Financial Services transaction processing time across controlled node profiles?

RQ2 - Operational impact: Does routing based on the best latency-prediction model improve end-to-end latency, throughput, SLA compliance, and simulated infrastructure cost?

RQ3 - Routing comparison: How does ML-based routing compare with Round Robin, Weighted Round Robin, Least-Loaded, and rule-based routing during normal and festival-surge traffic?

Optional RQ4 - Robustness: How robust is ML-based routing during node degradation and complete node failure?

7. Proposed Data Strategy

7.1 External Transaction Source

A PaySim-style synthetic mobile-money dataset is proposed as the external transaction source. Using a documented external distribution is preferable to relying only on an undocumented team-created random generator. However, the exact repository, version, provenance, and license must be verified before the dataset is adopted.

PaySim supplies transaction attributes and fraud-related fields. It does not supply transaction-processing latency, CPU demand, memory demand, or a validated workload label. Fraud status will not be treated as processing workload.

7.2 Measured FinCluster Execution Dataset

The latency target will be generated by executing sampled transactions through the FinCluster reference processing pipeline on controlled node profiles. The final research dataset will combine pre-routing transaction attributes, node-profile attributes, controlled runtime conditions, and measured service time.

PaySim row	Reference pipeline	Controlled node	Measured label	Model dataset
------------	--------------------	-----------------	----------------	---------------

8. FinCluster Reference Processing Pipeline

The following stages are provisional. Common validation will be shared, while transaction-type-specific fields and operations may add or skip processing branches. The exact pipeline must be mentor-reviewed before measurement.

1. Common request and schema validation
2. Transaction-ID and idempotency lookup
3. Required account, merchant, or agent lookup
4. Balance, limit, and transaction-rule evaluation
5. Optional risk-model inference or enhanced verification
6. Database transaction and ledger update
7. Audit-log write
8. Response generation

The study will execute real validation, Python logic, model inference, and test-database operations. It will not simulate workload by assigning arbitrary work units or inserting artificial sleep delays as the primary source of latency.

9. Target and Feature Definition

9.1 Primary Target

Primary model target

Measured service_time_ms

Service time covers processing from worker start to response completion. Queue waiting and routing-decision time are excluded from the regression target and measured separately during system evaluation.

9.2 Proposed Input Groups

Feature group	Examples	Important rule
Transaction features	type, amount, pre-transaction balances, time-step, valid derived indicators	Must be available before routing
Node features	CPU quota, memory limit, node profile, concurrency	Same columns across profiles; values differ
Runtime conditions	controlled load band, queue length at routing time	Include only when accurately recorded before execution
Supporting measurements	CPU time, database time, peak memory, stage count	Mainly for analysis; not automatically model inputs

Leakage review is required for: fraud labels, post-transaction balances, raw account identifiers, and any field generated after the outcome is known.

10. Controlled Measurement Protocol

- Run every sampled transaction on every selected node profile.
- Use the same pipeline version and software environment for all comparisons.
- Perform warm-up runs before measured repetitions.
- Repeat measurements and use the median service time as the stable target.
- Randomise run order to reduce time-order and thermal bias.
- Keep all repetitions of one transaction in the same data split.
- Record hardware, operating system, Docker limits, database configuration, dependency versions, and random seeds.

Provisional pilot protocol: 2 warm-up runs and 5 measured repetitions per transaction-node pair. The final count will be decided after pilot variance analysis.

11. Candidate Regression Models

Model	Reason for inclusion
Linear Regression	Interpretable linear baseline
Decision Tree Regressor	Transparent non-linear baseline
Random Forest Regressor	Bagging ensemble and current-project continuity
Extra Trees Regressor	Highly randomised tree ensemble
HistGradientBoosting Regressor	Efficient boosting baseline
XGBoost Regressor	Strong gradient-boosting candidate
CatBoost Regressor	Strong tabular candidate with categorical-feature support

12. Evaluation Design

12.1 Model-Level Evaluation

This evaluation answers whether a model can predict measured service time accurately.

- MAE - primary error metric
- RMSE - penalises large errors more strongly
- R-squared - variance explained
- Model inference latency
- Model size

12.2 System-Level Evaluation

This evaluation answers whether using the prediction inside the routing engine improves the complete system.

- Average end-to-end latency
- P95 latency
- Throughput
- Queue waiting time
- SLA-violation rate
- Simulated infrastructure cost
- Node utilisation

Two-layer research story

Layer 1: Find which model predicts transaction-node service time best.

Layer 2: Test whether the winner model creates better routing outcomes than traditional algorithms.

13. Routing Baselines and Scenarios

13.1 Routing Baselines

- Round Robin
- Weighted Round Robin
- Least-Loaded
- Rule-Based Routing
- Winner-Model ML Routing

13.2 Main Scenarios

- Normal traffic
- Festival-surge traffic

13.3 Optional Scenario

- Node degradation and complete node failure, subject to mentor approval and implementation feasibility.

14. Expected Contributions

- A reproducible measured transaction-node service-time dataset derived from an external mobile-money transaction distribution and a documented reference pipeline.
- A fair comparison of multiple regression models for node-aware latency prediction.
- Evidence showing whether model-level accuracy translates into measurable routing-system improvement.
- A controlled FinCluster framework for comparing ML-based and traditional routing under normal and surge traffic.

15. Scope and Limitations

- The transaction data are synthetic and privacy-safe, not real customer records.
- The processing pipeline is a FinCluster reference implementation, not a universal bank standard.
- Latency results are specific to the documented experimental host, software stack, database, and node profiles.
- The first model will target known controlled node profiles; unseen hardware requires calibration or retraining.
- No real financial settlement will occur.
- No verified experimental result is currently available.
- The research gap and novelty claim remain provisional until literature review is completed.

16. Immediate Day 2 Plan

1. Obtain mentor approval for the revised proposal and research questions.
2. Verify the exact PaySim source, version, provenance, and license.
3. Run the inspection notebook or script without training a model.

4. Draft the common and type-specific pipeline policy.
5. Create a small pilot pipeline before building the full measurement dataset.
6. Pilot Docker node profiles and measure run-to-run variance.
7. Freeze the dataset schema and experiment protocol before model implementation.

17. Approval Checklist

Item	Decision	Mentor notes
Primary node-aware latency-regression scope	Pending	
Reference-pipeline stages	Pending	
PaySim source/version/license	Pending	
Controlled node profiles	Pending	
Candidate regression models	Pending	
Normal and festival-surge scenarios	Pending	
Optional failure scenario	Pending	